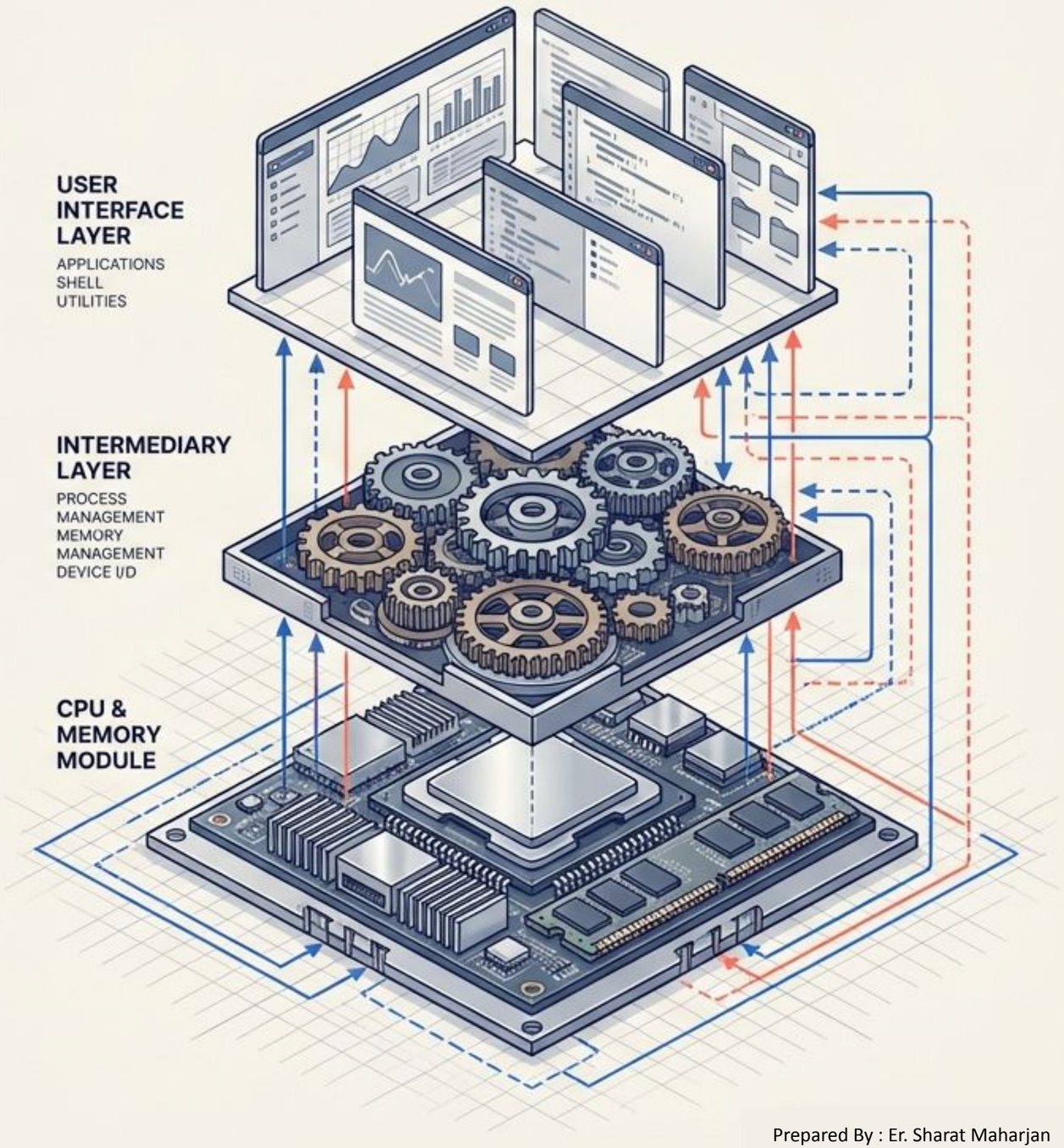
Operating System Overview

- **Unit 1:** Core Concepts & Evolution
- System Architectures & Types
- **Kernel** Anatomy & System Calls
- **Bachelor of Multimedia:** Technical Foundations



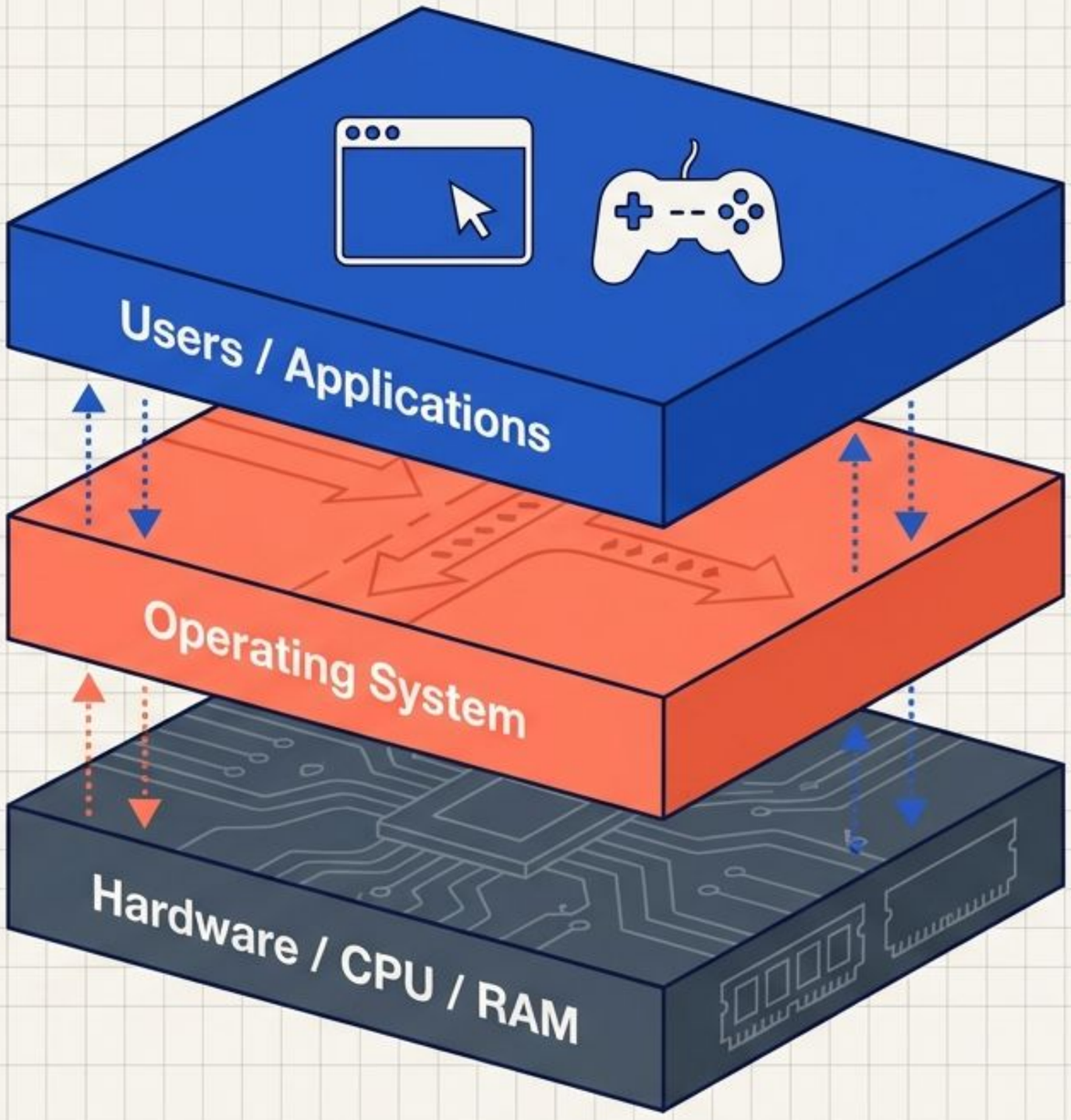
What is an Operating System?

The Ultimate Resource Manager
Allocates hardware and software resources.

The Intermediary
Bridges the gap between users/apps and raw computer hardware.

The Control Program
Acts as a traffic controller preventing conflicts.

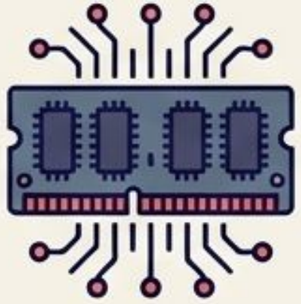
The Execution Environment
Provides a safe, isolated space for programs to run.



Four Fundamental Functions

Process Management

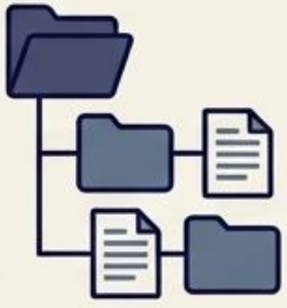
Creation, scheduling, and synchronization of CPU tasks.



Memory Management

RAM allocation/deallocation and virtual memory handling.

OS Core



File System Management

Hierarchical organization of data storage.

Device Management

Handling I/O operations and device drivers.

Two Views of the OS

User View

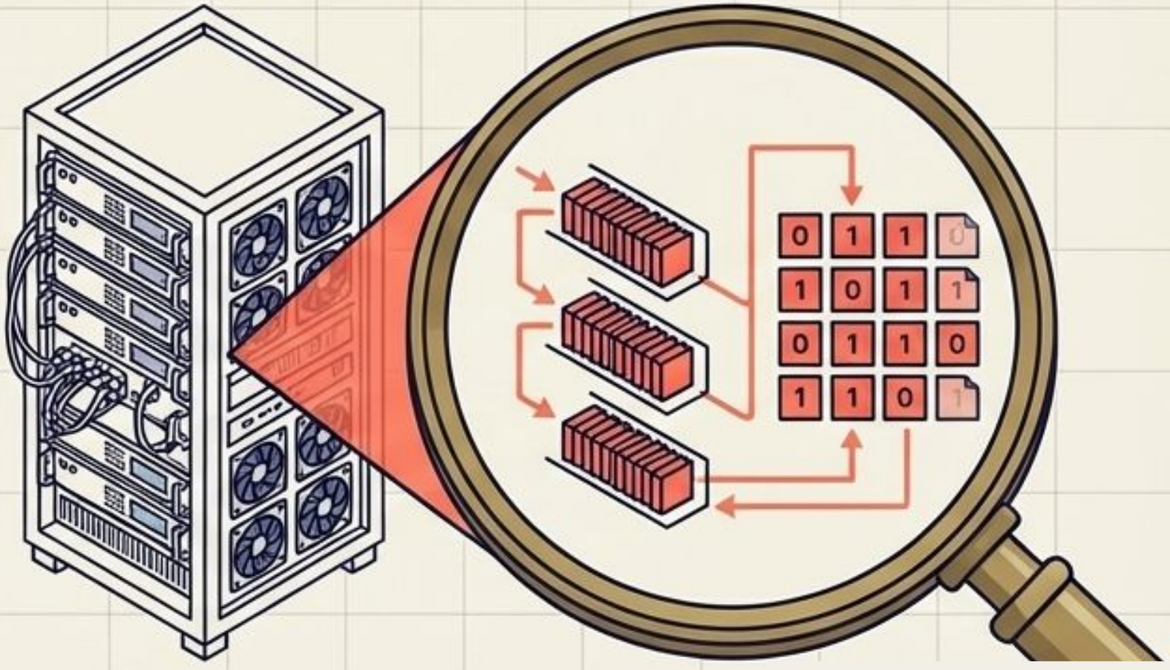
Focuses on interaction, UI/UX, and performance.

PC: GUI and responsiveness.
Mobile: Touch interface and battery optimization.
Server: Stability and uptime prioritized over UI.

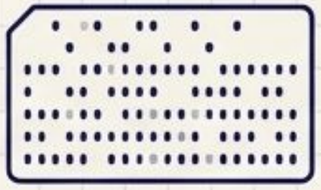
System View

Focuses on the OS as a machine manager.

Resource Allocator: Manages CPU scheduling, memory paging, and I/O bus access.
Control Program: Enforces security policies and prevents process errors.



Evolutionary Timeline



1940s-50s

No OS: Machine language, manual punch cards.

1950s-60s

Batch: Automated job sequencing (e.g., IBM OS/360).

1960s-70s

Time-Sharing: Concurrent user access (e.g., UNIX).

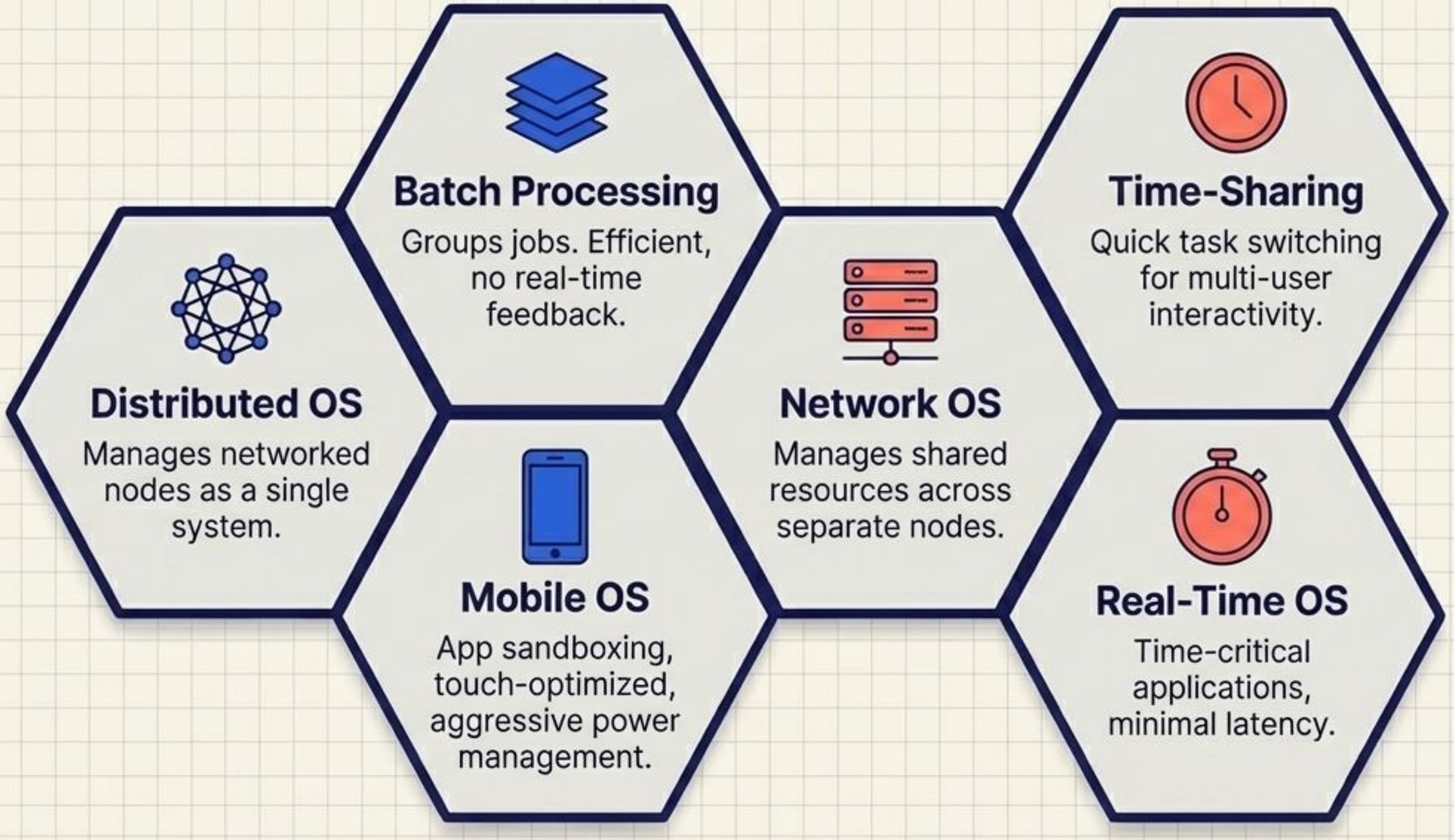
1980s-90s

Personal PCs: Rise of the GUI, user-friendly.

2000s-Present

Mobile/Distributed: Cloud, IoT, touch optimization.

OS Taxonomy & Types



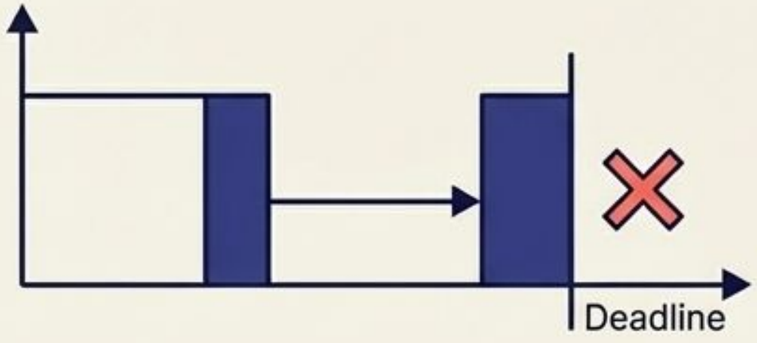
Real-Time OS: Hard vs. Soft

RTOS is designed for **time-critical** applications; zero tolerance for unacceptable delay.

Hard RTOS

Rule: Strict, fixed deadlines. Missing a deadline causes catastrophic failure.

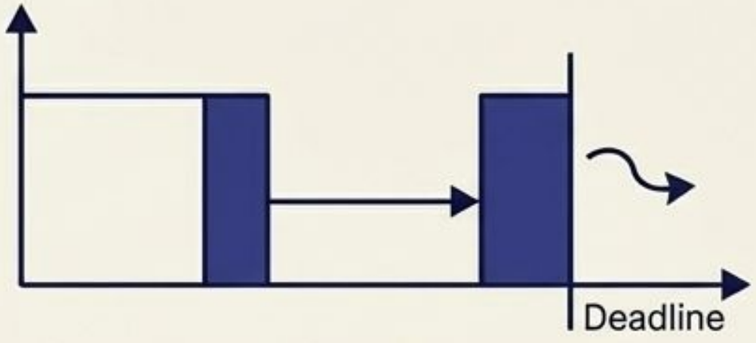
Examples: Medical pacemakers (FreeRTOS), Car airbags (SafeRTOS).



Soft RTOS

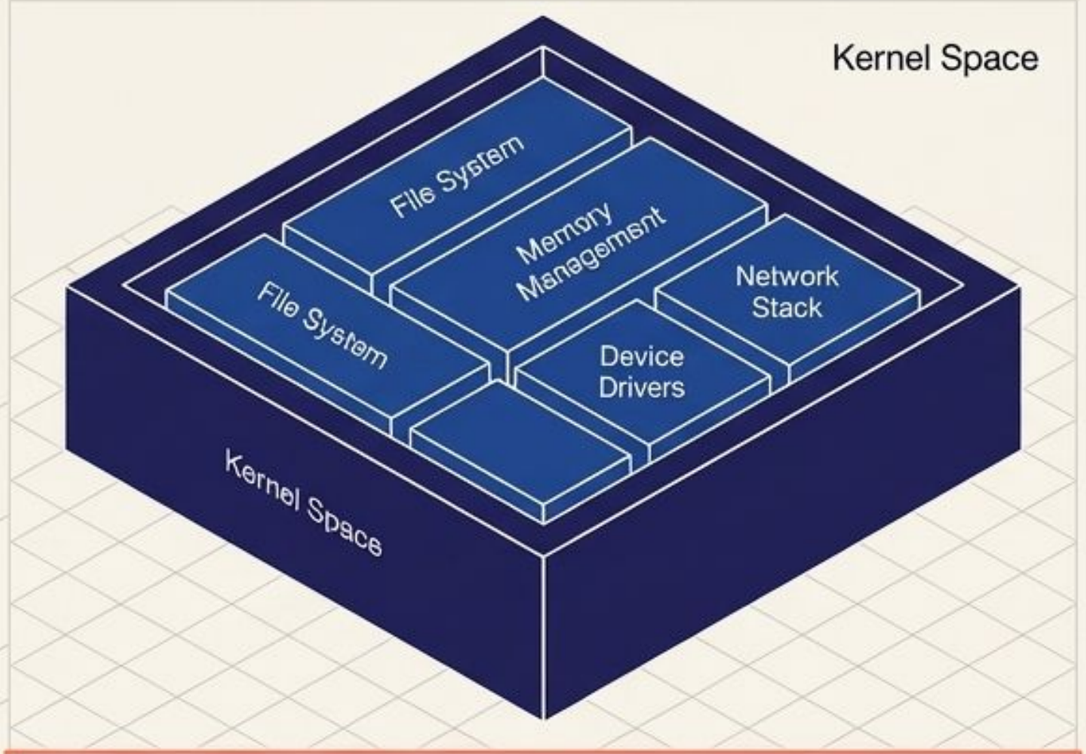
Rule: Deadlines are important, but missing them occasionally only reduces quality, not safety.

Examples: Video streaming (VxWorks), online gaming.



Structural Anatomy: Monolithic vs. Microkernel

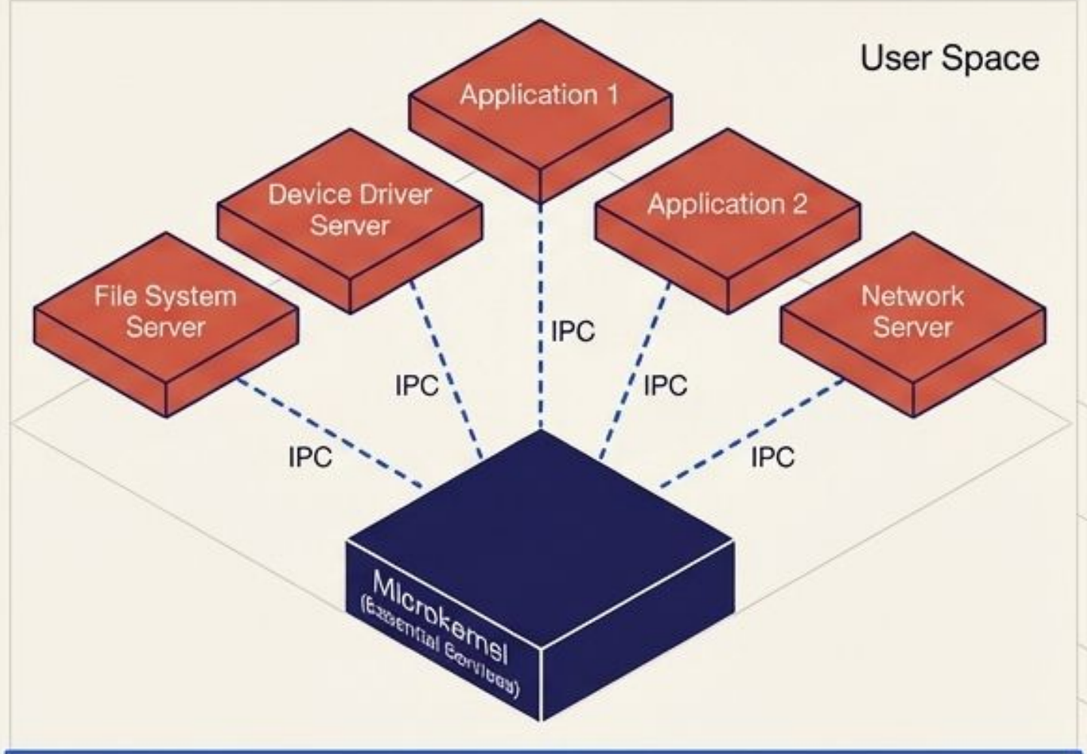
Monolithic Kernel



Concept: All core functions run as a single large program in Kernel Space.

Pros/Cons: Ultra-fast execution, but a single bug can crash the whole system.

Microkernel



Concept: Only essential services in kernel. Drivers/files run safely in User Space.

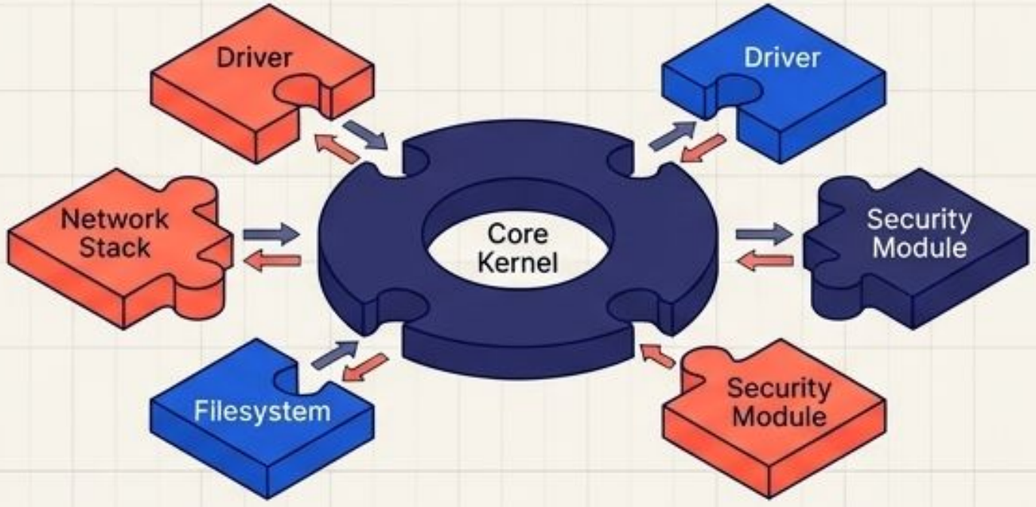
Pros/Cons: High security/isolation, but slower due to IPC message passing.

Advanced & Modern Architectures

Modular Structure (Modern Linux)

Combines monolithic speed with microkernel flexibility via Loadable Kernel Modules (LKMs).

Dynamically load/unload components without rebooting.

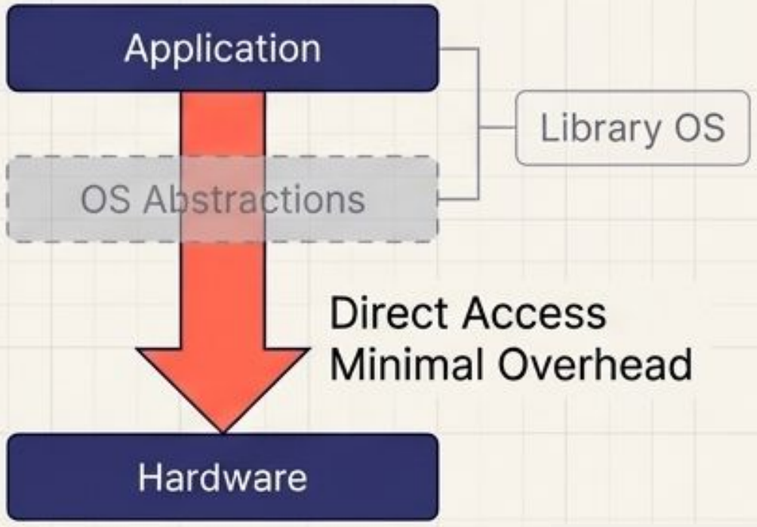


Exokernels (Research / Ultra-Performance)

Extreme minimalism. No abstractions.

Kernel only securely allocates raw hardware.

Apps link to a Library OS to build custom systems.



System Calls: The OS Bridge

Programmatic requests from user applications to the kernel for privileged operations.

- **Process Control:**
  - fork(), exit()
- **File Management:**
  - open(), read()
- **Device Management:**
  - ioctl(), write()
- **Inter-Process Comms:**
  - pipe(), shmget()

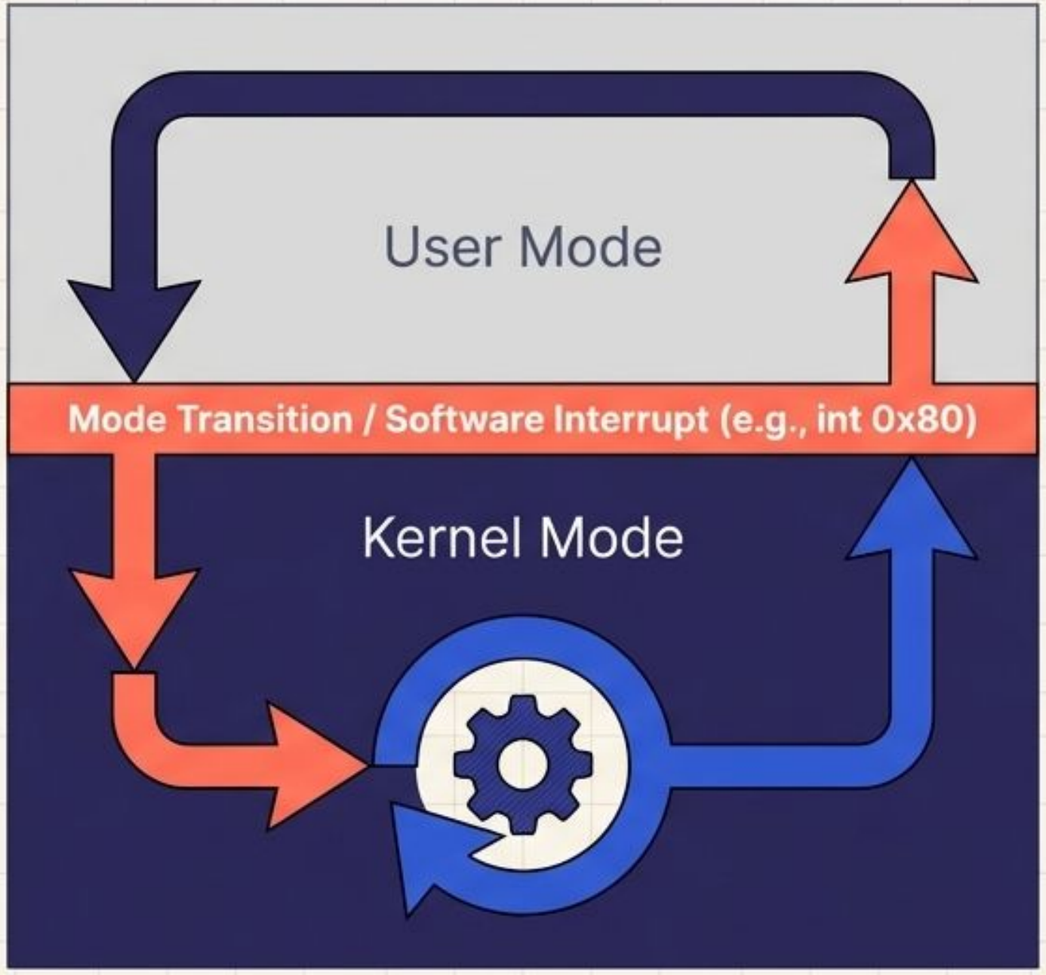
```
write(1, "Hello OS!", 9); // System call to print
```

System Call Execution: The Mode Switch

The secure process of transitioning from User Mode to Kernel Mode for privileged operations.

Step 1: Application Request
User program triggers a software interrupt.

Step 2: Mode Transition
CPU switches from User Mode (unprivileged) to Kernel Mode (privileged).

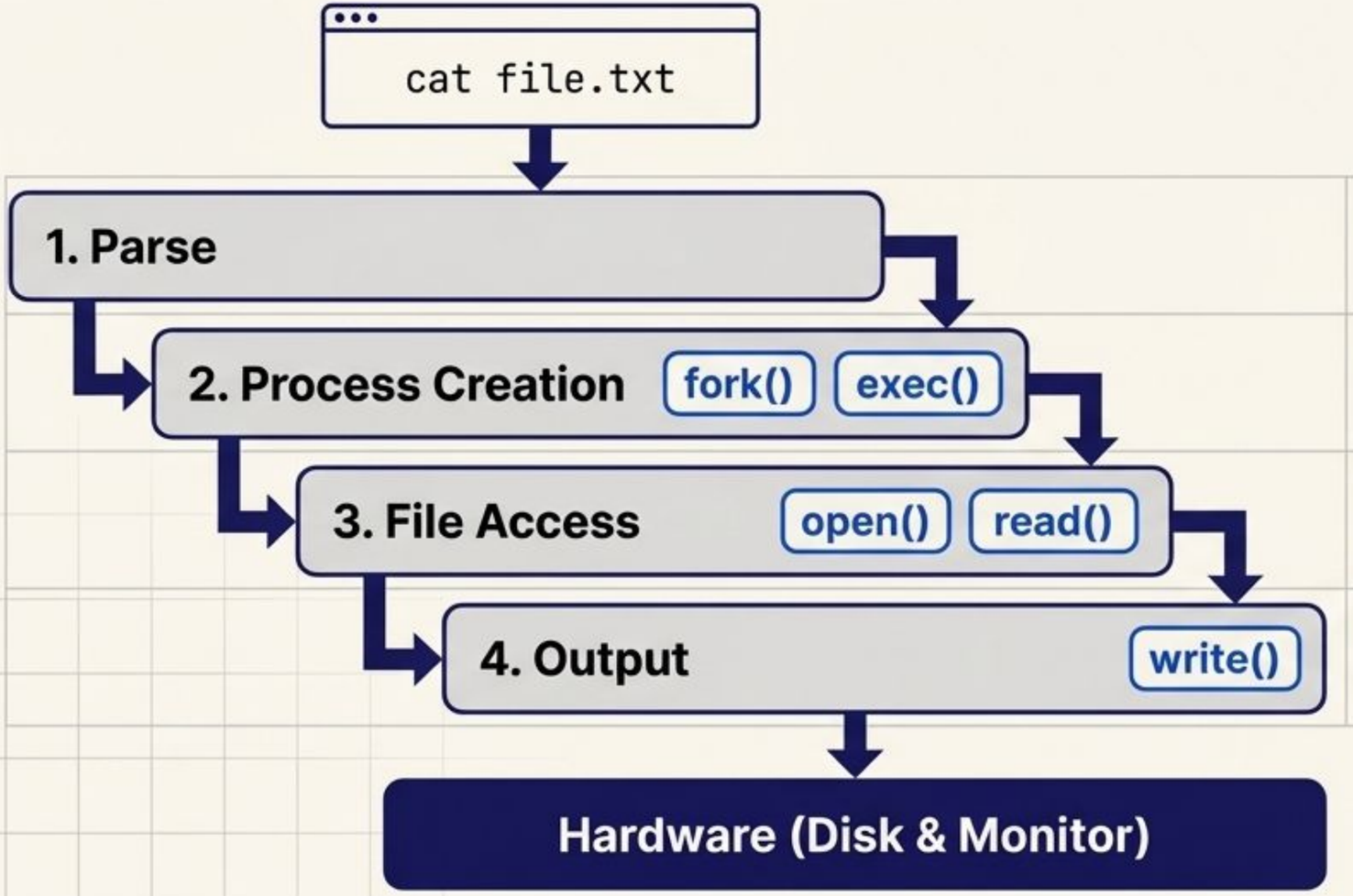


Step 4: Return
OS switches CPU back to User Mode and returns results.

Step 3: Kernel Execution
OS validates request and executes hardware instruction securely.

Applied Trace: Shell to System Call

Scenario: A user runs '`cat file.txt`' in the terminal.



1. Parse	Shell identifies program (**cat**) and argument (**file.txt**).
2. Process Creation (fork() exec())	Shell calls **fork()** to create a child, then **exec()** to load **cat**.
3. File Access (open() read())	Program calls **open()** to access the file and **read()** to pull contents.
4. Output (write())	Program calls **write()** to print contents to the screen.

The Outer Layer: Shell & System Programs

The Shell

A command-line interpreter bridging users and the OS. Accepts commands, translates to system calls.

- Bash (Linux default)
- Zsh (Auto-completion)
- PowerShell (Object-oriented)

System Programs

User-friendly utilities for managing system resources.

File Management: ls, cp, File Explorer

Process Management: top, kill, Task Manager

Networking: ping, netstat

The Open-Source OS Ecosystem

Definition: OS source code is publicly accessible, modifiable, and distributable.
Key Players: Linux, FreeBSD, Android

1. Transparency

Auditable code enables rapid bug fixes (e.g., Heartbleed).

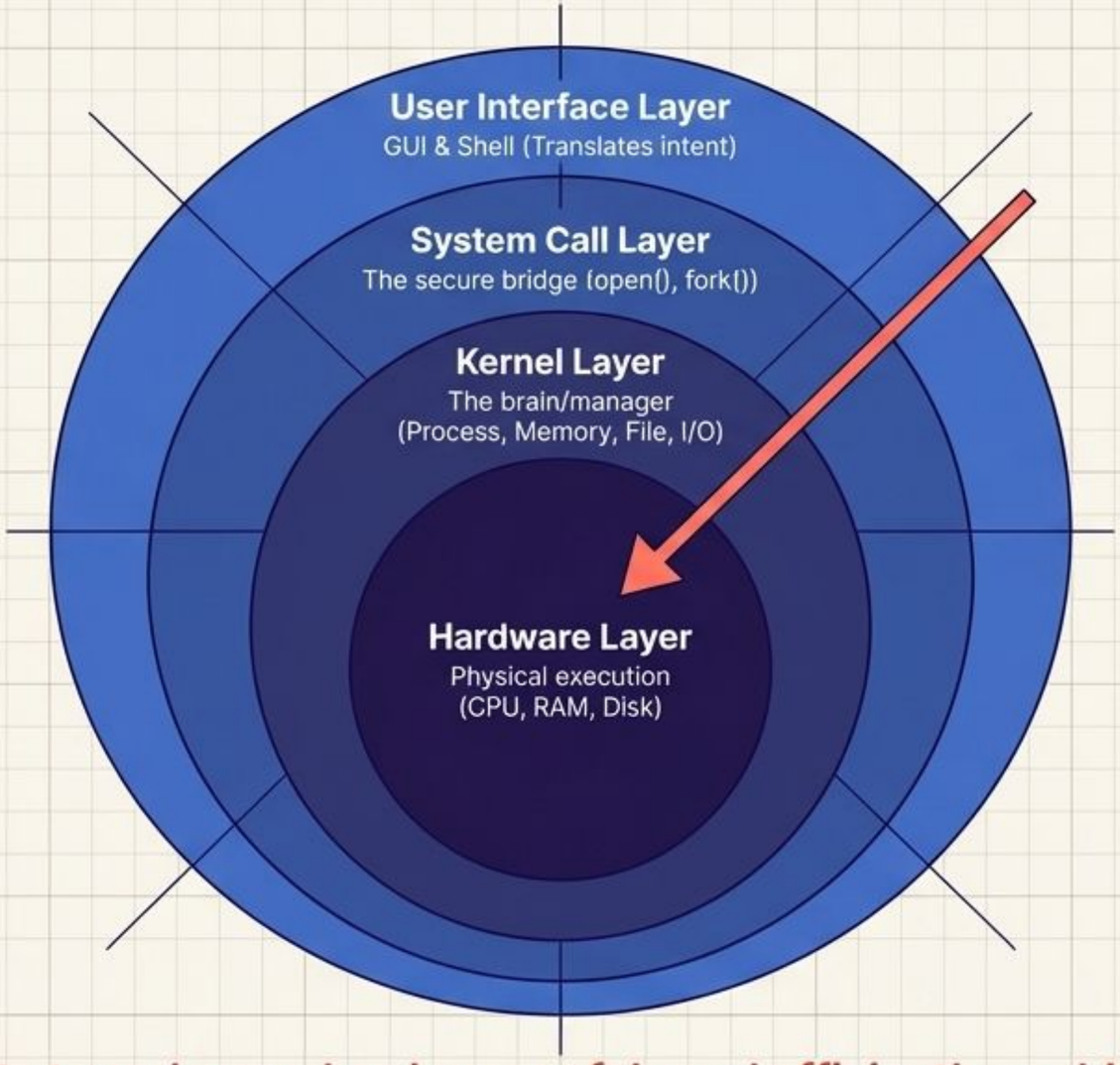
2. Community-Driven

Global developer base ensures constant updates.

3. No Vendor Lock-in

Avoids the constraints of proprietary corporate ecosystems.

Synthesis: The Complete OS Architecture



Core Insight: *"An OS exists to make raw hardware safely and efficiently usable by infinite applications."*